SOFTWARE ARCHITECTURE

Layered Approach to Solution Architecture

BEYOND BOUNDARIES

HOA SEN UNIVERSITY
WORLD CLASS EDUCATION

QS STARS RATING SYSTEM

ACBSP ACCREDITED

AUN-QA

MOET ACCREDITED
KIỂM ĐỊNH CHẤT LƯỢNG GIÁO DỤC QUỐC GIA

Layered Approach to Solution Architecture

- Layers in layered architecture
- Practical implementation of layered architecture in .NET Core

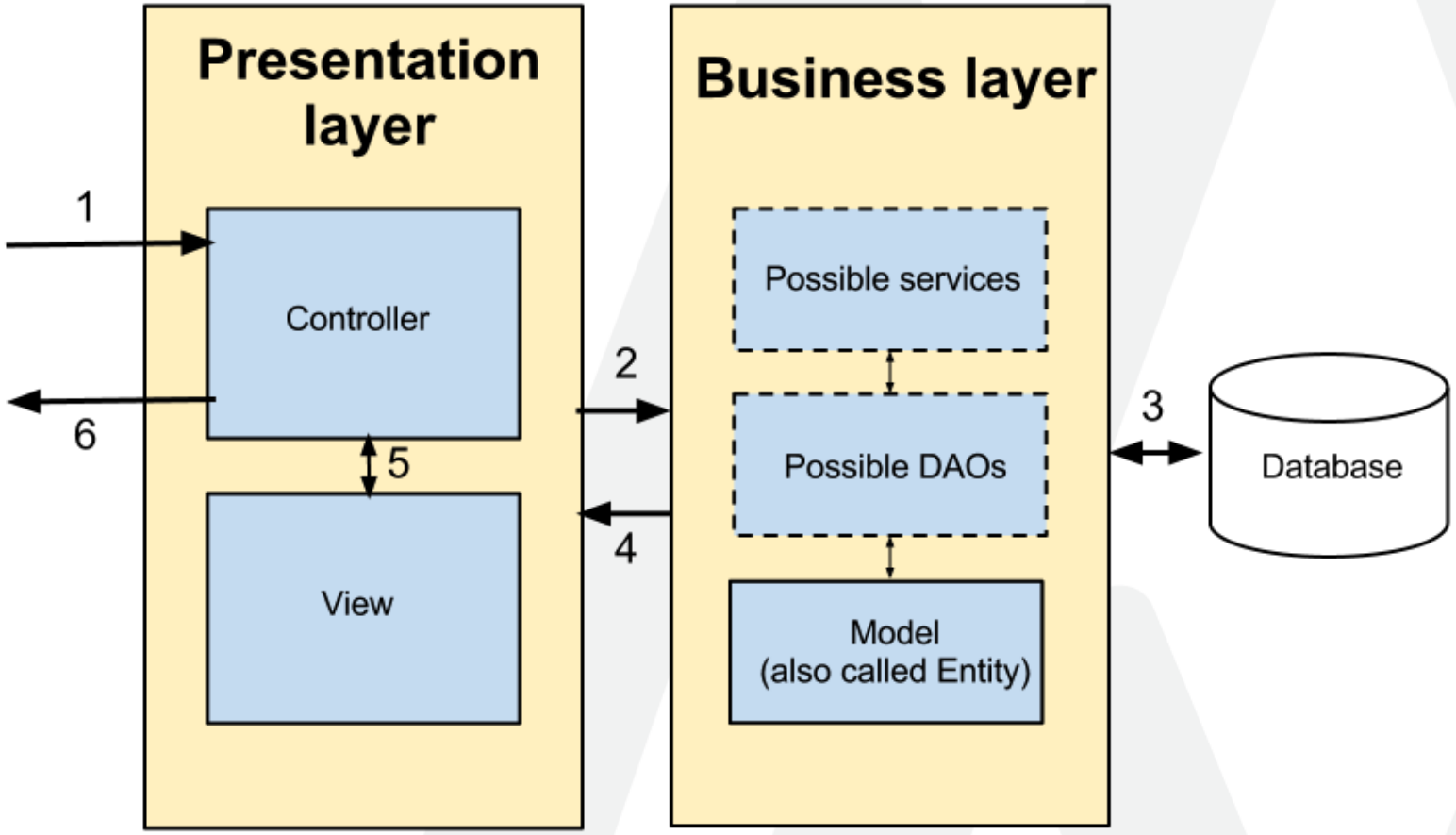
Layer vs Tier

- Read the article N-Tier / N-Layer Architecture at https://www.linkedin.com/pulse/n-tier-n-layer-architecture-swapnil-baxi/
- Share with your classmates what are different between N-Tier and N-Layer concepts

Layers in layered architecture

- An enterprise application consists of various layers.
- Each layer is **independent** and **fewer dependencies** on the other layer that makes it more pluggable and easier to maintain.
- Communication with other layers can be made through **interfaces** that encapsulate the logic or implementation details from the calling layer.
- Normally, business applications are n-tiered and logically divided into
  - The presentation layer
  - Service layer
  - Business layer
  - Data access layer

Layers in layered architecture



This Photo by Unknown Author is licensed under CC BY-SA

The benefits of layered architecture

- It is a loosely coupled system
- Teams can work on different layers, in parallel, with minimal dependencies on other teams
- Changes to any layer in terms of technology or business logic have little impact on the other layers
- Testing can be done easily

Presentation layer

- This layer contains all the **user interfaces** of an application.
- This is the **frontend layer** with which the user interacts.
- Keeping this layer separate facilitates upgrading the existing interfaces or design without making any changes to the backend layers.
- There are some architectural principles which should be followed when designing the presentation layer

Three principles of presentation layer design

- Our data should be separated from view and use patterns like MVC, MVVM, or MVP to address separation of concerns.
- Separate teams should work on the UI and business layer simultaneously.
- Minimize the application logic separate from the presentation layer

Note for presentation layer design

- The presentation layer should only contain forms, fields, validation, and related checks.
- All server-side logic should reside on the business layer.

Service layer

- The service layer is the main layer, which connects the presentation layer to the business layer.
- It exposes some methods through class libraries, Web APIs, or web services used by the presentation layer to perform certain operations and access data.
- Moreover, it also encapsulates the business logic, and exposes POST, GET, and other HTTP methods to perform certain functionality.
- There are few advantages of **keeping it web-based** instead of making it a shared library project.

Web-service vs Shared library

Web-service	Shared library
Any customer can access	No Internet connection is required
Code cannot be compiled	No serialization overhead
Easy upgrades or bug fixes	No risk of forgery
Scalable	Higher performance
Centralized deployments	No down time of service impacting the consumer

Service layer

- The service layer can also contain methods that do not call the business layer, but are helper methods or common methods
- These methods are used by the presentation layer and other consuming parties to the achieve desired functionality.
- For example, the service layer can contain a method that takes a list of strings as a parameter and returns a CSV (comma separated value) string where this operation does not require any backend database operation.

Service layer

- Securing services is highly recommended when building enterprise applications
- With ASP.NET, this can easily be achieved by authenticating the user using the ASP.NET security mechanism, protecting resources imperatively through the Authorize attribute or custom policies.
- The service layer is the primary layer that exposes data to third-party sources.
- Therefore, logging and transactions should be properly addressed.

Approaches to implement the Service layer

- .NET stack
  - Web API
  - Service Stack
  - WCF
  - POCOs (POrtable Components)
- .NET Core
  - Web API
  - Service Stack
  - POCOs libraries.

Business layer

- Is the core layer in the layered architecture.
- It contains the **actual logic** of the application and manages the events that trigger from the presentation layer.
- There are different approaches when building the business layer and every approach is dependent on the scope and timeline of the project.

Designing or Architecting the business layer

- Transaction Script pattern
- Table Module pattern
- Active Record pattern
- Domain Driven Design (DDD)

Transaction Script pattern

- The business layer provides simple public methods to perform a particular functionality.
- This is a procedural approach, where each method represents the corresponding operation taken on the presentation layer.
- For example, the SubmitVendor action may have a corresponding method in the business layer such as CreateVendor and so on.

Table Module pattern

- Each class represents one entity, used to perform CRUD (create, read, update, and delete) operations on the table.
- Considering the .NET stack, we mostly used the DataSet and DataTable objects, that is bound to the table in the backend database.

Active Record pattern

- Each instance of the class represents a database row.
- If we have an instance of the class that is representing a particular row, we can use that instance to perform CRUD operation on that row and this can be implemented using Entity Framework or other ORM (Object Relational Mapper) tools.
- If you have noticed in the Entity Framework, every class is bound to a consequent table, and contains properties that are tightly bound to the table columns.
- With this pattern, each POCO class represents a database row and contains a key and other properties to represent that row.

Domain Driven Design (DDD)

- Domain Driven Design is a complex pattern, but not tightly coupled with the backend database.
- In this approach, classes are structured in such a way as to represent the entity of the domain and not the table.
- For example
  - If we are going with the DDD pattern, we may have three tables--User, Address, and Profile, which represent the company's employee as a domain entity.
  - This is a good approach; however, it takes sufficient time to understand and structure the domain entities. It represents the business language and hides the logical details from the presentation layer.

Discussion

- Read an article named What is Code-First? at https://www.entityframeworktutorial.net/code-first/what-is-code-first.aspx
- What are the main characteristics of this approach?
- In your opinion, what are the benefits of Code-First?

Data access layer

- The DAL (data access layer) is the core layer that interacts with the backend database.
- There are certain core classes we develop, which communicates with the database, manage connections and execute the CRUD operations.
- Certain patterns can be implemented to minimize the redundant effort of opening and closing connections and applying transactions that can be encapsulated through a simple interface and used by the business layer to perform CRUD operations.
- The most widely used pattern for this layer is the **Repository pattern**

Objectives of layered architecture

- To provide separation between each layer.
- Each layer should be designed in such a way that it has very little, or no impact on the other layers.
- There are different techniques to achieve **decoupling between layers and interfaces** are one of the techniques that are primarily used.
- For example, there are cases where we may want to change the database engine in the DAL and instead of using SQL Server, we can choose Oracle. Or, in the case of the UI, we may want to change the Web Forms framework for an MVC framework.
- Good architecture always provide this decoupl**ing and reduces dependencies.**

Reading

- **Design the infrastructure persistence layer, 21-Sep-2022,** https://learn.microsoft.com/en-us/dotnet/architecture/microservices/microservice-ddd-cqrs-patterns/infrastructure-persistence-layer-design
- **Implementing the Repository and Unit of Work Patterns in an ASP.NET MVC Application (9 of 10)**, 07-Jan-2022, https://learn.microsoft.com/en-us/aspnet/mvc/overview/older-versions/getting-started-with-ef-5-using-mvc-4/implementing-the-repository-and-unit-of-work-patterns-in-an-asp-net-mvc-application

Summary

- You have learnt some best practices when designing an n-layer architecture.
- We explored the benefits of keeping entities and other helper classes in the common layer so that they can be used throughout the layers.
- In practices, we will develop a few controllers to show how the business managers can be injected and how logging and exception handling can be done. In the presentation layer, we configured Angular and designed a frontend framework to invoke HTTP services, display messages, and view models to carry data.

Q&A

This Photo by Unknown Author is licensed under CC BY-NC-ND